

Lab 7: User Frames, Offsets & Structured Programs

EE0849 Introduction to Robotics — Roboguide Simulation

Basri Erdogan

Table of contents

0.1 Objectives	2
1 Part 1: Roboguide Workcell Setup (15 min)	3
1.1 Creating the Cell	3
1.2 Adding a Workbench (Fixture)	3
1.3 Adding Parts	3
2 Part 2: Tool Frame Calibration (15 min)	4
2.1 Why Calibrate the Tool?	4
2.2 Direct Entry Method	4
2.3 Verifying the Tool Frame	4
3 Part 3: User Frame Calibration (20 min)	5
3.1 Setting Up User Frame 1	5
3.2 Verifying the User Frame	5
4 Part 4: Offset-Based Pick and Place (25 min)	6
4.1 Teaching the Base Positions	6
4.2 Writing a Simple Pick-and-Place Program	6
4.3 Adding Offsets with Position Registers	6
5 Part 5: Multi-Program Architecture (20 min)	8
5.1 Splitting into Subprograms	8
6 Part 6: User Frame Switching (20 min)	10
6.1 Setting Up a Second Workbench Position	10
6.2 Calibrating User Frame 2	10
6.3 Running the Same Program at Both Positions	10
7 Part 7: Simple Palletizing (20 min)	12
7.1 Grid Placement with Offsets	12
8 Quick Reference	14
9 Completion Checklist	14

Lab Environment

This lab is performed entirely in **FANUC Roboguide** on the lab computers. No physical robot is used. Roboguide provides a full digital twin of the LR Mate 200iD — the same controller, the same TP language, and the same menus you use on the real pendant.

Prerequisites

You should be comfortable with:

- Jogging the robot and teaching positions (Lab 5)
- Writing basic TP programs with J/L motion (Lab 5–6)
- Registers (R[]) and position registers (PR[]) (Lab 6)

0.1 Objectives

In this lab you will:

1. Create a workcell in Roboguide with a workbench and parts
2. Calibrate a **Tool Frame** (UTOOL) using the 3-point method
3. Calibrate a **User Frame** (UFRAME) using the 3-point method
4. Write a pick-and-place program using **position register offsets**
5. Build a **multi-program architecture** (MAIN □ PICK □ PLACE)
6. Switch between **two user frames** for different workbench positions
7. Implement a simple **palletizing** routine with MOD/DIV

1 Part 1: Roboguide Workcell Setup (15 min)

1.1 Creating the Cell

Roboguide Steps

1. Launch Roboguide and select **File** ▢ **New Cell**
2. Choose **HandlingPRO** as the application
3. Select robot model: **LR Mate 200iD**
4. Accept default controller software version
5. Click **Finish** to create the cell

1.2 Adding a Workbench (Fixture)

Adding Objects

1. In the Cell Browser (left panel), right-click **Fixtures** ▢ **Add Fixture**
2. Select **Box** shape
3. Set dimensions: 400 mm × 300 mm × 50 mm (a tabletop)
4. Position it within the robot's reach:
 - X: 400 mm, Y: 0 mm, Z: 0 mm (in front of the robot base)
5. Rename it to Workbench_A

1.3 Adding Parts

Adding Parts

1. Right-click **Parts** ▢ **Add Part**
2. Select **Box** shape: 30 mm × 30 mm × 20 mm
3. Place it on the workbench surface (Z = 50 mm + part half-height)
4. Add 2–3 more parts at different positions on the workbench

Exercise 1: Workcell Setup

Take a screenshot of your completed workcell showing the robot, workbench, and parts. Record the workbench position you used:

Workbench X: _____ Y: _____ Z: _____

2 Part 2: Tool Frame Calibration (15 min)

2.1 Why Calibrate the Tool?

In Roboguide, you can attach a virtual gripper to the robot. Even in simulation, the controller needs to know where the TCP is relative to the flange — otherwise linear moves and jogging in TOOL coordinates will reference the wrong point.

2.2 Direct Entry Method

For this lab, we will use the direct entry method since we know the exact gripper dimensions from the CAD model.

Setting Up UTOOL

1. On the virtual teach pendant: **MENU** ▢ **SETUP** ▢ **Frames**
2. Select **Tool Frame**
3. Choose frame number **1**
4. Select method: **Direct Entry**
5. Enter the TCP offset (example for a simple gripper):
 - X: 0.0 mm, Y: 0.0 mm, Z: 100.0 mm
 - W: 0.0°, P: 0.0°, R: 0.0°
6. Press **ENTER** to confirm

2.3 Verifying the Tool Frame

Verification

1. Set UTOOL_NUM = 1 on the pendant
2. Press **COORD** until you see **TOOL** in the top-right corner
3. Jog the robot along each axis — the TCP (tool tip) should move, not the flange
4. Rotate the tool — it should pivot around the TCP, not the flange center

Exercise 2: Tool Frame Verification

With UTOOL[1] active, jog the robot to any position and record the Cartesian display:

X: _____ Y: _____ Z: _____
W: _____ P: _____ R: _____

Now switch to UTOOL[0] (flange, no tool) and record the same robot position:

X: _____ Y: _____ Z: _____

What is the difference in Z between the two readings? Does it match your TCP offset?

3 Part 3: User Frame Calibration (20 min)

3.1 Setting Up User Frame 1

We will calibrate a user frame at the corner of the workbench using the 3-point method.

Three Points Define the Frame

1. **Origin Point** — the corner of the workbench (frame origin)
 2. **X-Direction Point** — a point along the long edge (defines X axis)
 3. **Y-Plane Point** — a point along the short edge (defines XY plane)
- The controller computes ${}^{World}T_{User}$ from these three points.

Calibrating UF[1]

1. **MENU** ▢ **SETUP** ▢ **Frames** ▢ **User Frame**
2. Select frame number **1**
3. Select method: **Three Point**
4. **Orient Origin Point:**
 - Jog the TCP to the front-left corner of the workbench surface
 - Press **SHIFT + RECORD**
5. **X Direction Point:**
 - Jog the TCP along the long edge (positive X direction)
 - Press **SHIFT + RECORD**
6. **Y Direction Point:**
 - Jog the TCP along the short edge (positive Y direction)
 - Press **SHIFT + RECORD**
7. The frame is calculated automatically

3.2 Verifying the User Frame

Verification

1. Set `UFRAME_NUM = 1`
2. Press **COORD** until you see **USER** coordinates
3. Jog along X — the TCP should move along the workbench long edge
4. Jog along Y — the TCP should move along the short edge
5. Jog along Z — the TCP should move straight up from the surface
6. Jog the TCP to the origin corner — the display should read approximately X=0, Y=0, Z=0

Exercise 3: User Frame Verification

With UF[1] active, jog the TCP to the workbench origin corner. Record the displayed coordinates:

X: _____ Y: _____ Z: _____

Are they close to (0, 0, 0)? If not, what might have caused the offset?

Now jog to the opposite corner of the workbench and record:

X: _____ Y: _____ Z: _____

Do X and Y approximately match the workbench dimensions (400 mm × 300 mm)?

4 Part 4: Offset-Based Pick and Place (25 min)

4.1 Teaching the Base Positions

With **UFRAME[1]** and **UTOOL[1]** active, teach the following positions:

Position	Description	Approximate UF[1] Coordinates
P[1]	Pick approach (above part)	X=50, Y=50, Z=60
P[2]	Pick point (at part surface)	X=50, Y=50, Z=20
P[3]	Place approach (above target)	X=300, Y=200, Z=60
P[4]	Place point (at surface)	X=300, Y=200, Z=20

Approach Height

The approach points (P[1], P[3]) are directly above the pick/place points. The Z difference is the approach height — the robot descends vertically for the final move. This prevents collisions with surrounding objects.

4.2 Writing a Simple Pick-and-Place Program

Create a new program called **PICK_PLACE**:

```
/PROG PICK_PLACE
  1: UFRAME_NUM=1 ;
  2: UTOOL_NUM=1 ;
  3: ! --- PICK ---
  4: J P[1] 100% FINE ;      ! Approach pick
  5: L P[2] 100mm/sec FINE ; ! Descend to part
  6: DO[1:Gripper]=ON ;
  7: WAIT .30(sec) ;
  8: L P[1] 500mm/sec FINE ; ! Retract
  9: ! --- PLACE ---
 10: J P[3] 100% FINE ;      ! Approach place
 11: L P[4] 100mm/sec FINE ; ! Descend to surface
 12: DO[1:Gripper]=OFF ;
 13: WAIT .20(sec) ;
 14: L P[3] 500mm/sec FINE ; ! Retract
/END
```

Creating the Program

1. **SELECT**  **F2 [CREATE]**
2. Enter program name: PICK_PLACE
3. Teach positions P[1]–P[4] by jogging and pressing **SHIFT + RECORD**
4. Type the motion and I/O instructions
5. Run in **single-step** mode first (**STEP** key)

4.3 Adding Offsets with Position Registers

Now modify the program to use offsets instead of separately taught approach points:

```
/PROG PICK_PLACE_V2
  1: UFRAME_NUM=1 ;
```

```

2: UT00L_NUM=1 ;
3: ! Define approach offset (40mm above)
4: PR[10,1]=0 ;
5: PR[10,2]=0 ;
6: PR[10,3]=40.0 ;
7: ! --- PICK ---
8: L P[1] 500mm/sec FINE Offset,PR[10] ;    ! Approach
9: L P[1] 100mm/sec FINE ;                  ! Pick
10: DO[1:Gripper]=ON ;
11: WAIT .30(sec) ;
12: L P[1] 500mm/sec FINE Offset,PR[10] ;    ! Retract
13: ! --- PLACE ---
14: L P[2] 500mm/sec FINE Offset,PR[10] ;    ! Approach
15: L P[2] 100mm/sec FINE ;                  ! Place
16: DO[1:Gripper]=OFF ;
17: WAIT .20(sec) ;
18: L P[2] 500mm/sec FINE Offset,PR[10] ;    ! Retract
/END

```

Why Offsets?

In V2, we only need **two** taught points (P[1] pick, P[2] place) instead of four. The approach/retract height is computed from a single offset register. To change the approach height for all moves, just change PR[10,3] — one number instead of re-teaching two positions.

Exercise 4: Pick and Place with Offsets

1. Create PICK_PLACE_V2 in Roboguide
2. Teach P[1] at a part location and P[2] at a place location
3. Run the program and verify the approach/retract motion
4. Change PR[10,3] from 40.0 to 80.0 and re-run — what changes?

Observation when approach height doubled:

5 Part 5: Multi-Program Architecture (20 min)

5.1 Splitting into Subprograms

Real applications separate pick and place into independent subprograms called from a main loop.

Create three programs:

MAIN:

```
/PROG MAIN
  1: UFRAME_NUM=1 ;
  2: UTOOL_NUM=1 ;
  3: ! Approach offset
  4: PR[10,1]=0 ; PR[10,2]=0 ; PR[10,3]=40.0 ;
  5: LBL[1] ;
  6: CALL PICK ;
  7: CALL PLACE ;
  8: JMP LBL[1] ;
/END
```

PICK:

```
/PROG PICK
  1: J P[1] 100% CNT50 ; ! Travel to pick zone
  2: L P[2:Pick] 500mm/sec FINE Offset,PR[10] ; ! Approach
  3: L P[2:Pick] 100mm/sec FINE ; ! Pick
  4: DO[1:Gripper]=ON ;
  5: WAIT .30(sec) ;
  6: L P[2:Pick] 500mm/sec FINE Offset,PR[10] ; ! Retract
/END
```

PLACE:

```
/PROG PLACE
  1: J P[1] 100% CNT50 ; ! Travel to place zone
  2: L P[2:Place] 500mm/sec FINE Offset,PR[10] ; ! Approach
  3: L P[2:Place] 100mm/sec FINE ; ! Place
  4: DO[1:Gripper]=OFF ;
  5: WAIT .20(sec) ;
  6: L P[2:Place] 500mm/sec FINE Offset,PR[10] ; ! Retract
/END
```

Note on Positions

P[1] and P[2] in PICK are **different** from P[1] and P[2] in PLACE — each program has its own local position table. PR[10] is **shared** across all programs (global register).

Exercise 5: Multi-Program Architecture

1. Create the three programs (MAIN, PICK, PLACE)
2. Teach positions in each subprogram
3. Run MAIN and observe the loop
4. Stop the program after 2–3 cycles (**HOLD**)

What happens if you change the approach offset PR[10,3] in MAIN? Does it affect both PICK and PLACE?

Why are UFRAME_NUM and UTOOL_NUM set in MAIN rather than in each subprogram?

6 Part 6: User Frame Switching (20 min)

6.1 Setting Up a Second Workbench Position

Now simulate moving the workbench to a different location.

Moving the Workbench in Roboguide

1. In the Cell Browser, select Workbench_A
2. Note the current position (Position A)
3. Change the fixture position to a new location:
 - X: 300 mm, Y: 200 mm, Z: 0 mm (shifted left and forward)
4. This is now Position B

6.2 Calibrating User Frame 2

Calibrating UF[2]

1. **MENU ▢ SETUP ▢ Frames ▢ User Frame**
2. Select frame number **2**
3. Select method: **Three Point**
4. Touch the same three reference features on the moved workbench:
 - Origin: front-left corner
 - X-direction: along long edge
 - Y-plane: along short edge
5. The frame is calculated at the new world position

6.3 Running the Same Program at Both Positions

Modify MAIN to run on both positions:

```
/PROG MAIN_DUAL
  1: UTOOL_NUM=1 ;
  2: PR[10,1]=0 ; PR[10,2]=0 ; PR[10,3]=40.0 ;
  3: !
  4: ! --- Work at Position A ---
  5: UFRAME_NUM=1 ;
  6: CALL PICK ;
  7: CALL PLACE ;
  8: !
  9: ! --- Work at Position B ---
 10: UFRAME_NUM=2 ;
 11: CALL PICK ;
 12: CALL PLACE ;
/END
```

The PICK and PLACE subprograms are **unchanged** — the same relative coordinates resolve to different world locations depending on the active user frame.

Exercise 6: User Frame Switching

1. Move the workbench in Roboguide to Position B
2. Calibrate UF[2] using the 3-point method
3. Move the workbench back to Position A (verify UF[1] still works)
4. Create MAIN_DUAL and run it

Record the world coordinates of P[2:Pick] as seen from each frame:

UF[1] – X: _____ Y: _____ Z: _____

UF[2] – X: _____ Y: _____ Z: _____

Are the relative coordinates (in user frame) the same? Are the world coordinates different?

Exercise 7: Frame Mismatch Experiment

Try this experiment to see the controller's safety check:

1. Open the PICK subprogram (positions were taught with UF[1])
2. In MAIN, change UFRAME_NUM=1 to UFRAME_NUM=3 (an uncalibrated or unrelated frame)
3. Try to run the program

What error message does the teach pendant display?

Why does FANUC prevent this instead of just running with wrong positions?

7 Part 7: Simple Palletizing (20 min)

7.1 Grid Placement with Offsets

Create a palletizing program that places parts in a 3x2 grid using a single taught pallet origin and computed offsets.

```

/PROG PALLET
  1: UFRAME_NUM=1 ;
  2: UT00L_NUM=1 ;
  3: PR[10,1]=0 ; PR[10,2]=0 ; PR[10,3]=40.0 ; ! Approach offset
  4: R[1]=0 ; ! Part counter
  5: LBL[1] ;
  6: CALL PICK ;
  7: ! Calculate grid position
  8: R[2]=R[1] MOD 3 ; ! Column (0, 1, 2)
  9: R[3]=R[1] DIV 3 ; ! Row (0, 1)
 10: PR[11,1]=R[2]*50.0 ; ! X = col * 50mm pitch
 11: PR[11,2]=R[3]*50.0 ; ! Y = row * 50mm pitch
 12: PR[11,3]=0 ;
 13: ! Place at pallet origin + grid offset + approach
 14: J P[1:Pallet_App] 100% CNT50 ;
 15: L P[2:Pallet_Origin] 500mm/sec FINE Offset,PR[11] ;
 16: DO[1:Gripper]=OFF ;
 17: WAIT .20(sec) ;
 18: L P[2:Pallet_Origin] 500mm/sec CNT50 Offset,PR[10] ;
 19: R[1]=R[1]+1 ;
 20: IF R[1]<6, JMP LBL[1] ; ! 3x2 = 6 parts
/END

```

MOD and DIV

MOD (modulo) gives the remainder: $5 \bmod 3 = 2$ (column index).

DIV (integer division) gives the quotient: $5 \div 3 = 1$ (row index).

Together they convert a linear counter into row/column indices.

Exercise 8: Palletizing

1. Create the PALLET program
2. Teach P[1] as an approach point above the pallet area
3. Teach P[2] as the pallet origin (where part 0 goes)
4. Run the program and observe the 3×2 grid placement

Sketch the grid layout and label each part position (0–5) with its (column, row) index:

What would you change to make a 4×3 grid with 40 mm pitch?

8 Quick Reference

Command	Description
UFRAME_NUM= <i>n</i>	Select user frame <i>n</i> (0 = world)
UTOOL_NUM= <i>n</i>	Select tool frame <i>n</i> (0 = flange)
PR[<i>n</i>]=LP0S	Store current Cartesian pose in PR[<i>n</i>]
PR[<i>n</i> ,1] through PR[<i>n</i> ,6]	Access X, Y, Z, W, P, R components
L P[<i>n</i>] speed FINE Offset, PR[<i>m</i>]	Move to P[<i>n</i>] shifted by PR[<i>m</i>]
CALL prog_name	Call a subprogram
R[<i>n</i>] MOD <i>k</i>	Remainder of R[<i>n</i>] / <i>k</i>
R[<i>n</i>] DIV <i>k</i>	Integer quotient of R[<i>n</i>] / <i>k</i>

9 Completion Checklist

- ☐ Workcell created with workbench and parts
- ☐ UTOOL[1] calibrated and verified
- ☐ UF[1] calibrated and verified (jogging in user coordinates)
- ☐ Pick-and-place with offsets working
- ☐ Multi-program architecture (MAIN, PICK, PLACE) running
- ☐ UF[2] calibrated for second workbench position
- ☐ Same program runs correctly at both positions
- ☐ Frame mismatch error observed and explained
- ☐ Palletizing routine places parts in 3×2 grid